SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	M. SANA FATHIMA	**Reg. No.:**	192371082
Date:	30/07/2026	**Duration:**	60 minutes

Code of Conduct
I __M. SANA FATHIMA (192371082)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: ___M .SANA FATHIMA___

Annexure A – Architecture Design Assignment

Title: Smart E-commerce Order Fulfillment Platform

System Requirement Analysis

- The Smart E-commerce Order Fulfillment Platform is designed to automate the complete order fulfillment process for e-commerce businesses. The system manages customer orders, inventory, warehouse allocation, payment processing, logistics, and delivery tracking.
- The main objective is to improve operational efficiency, reduce delivery delays, optimize inventory utilization, and enhance customer satisfaction. Functional requirements include customer registration, product browsing, order placement, payment processing, warehouse allocation, inventory management, order tracking, customer notifications, and report generation.
- Non-functional requirements include high performance, security, reliability, scalability, maintainability, and availability.

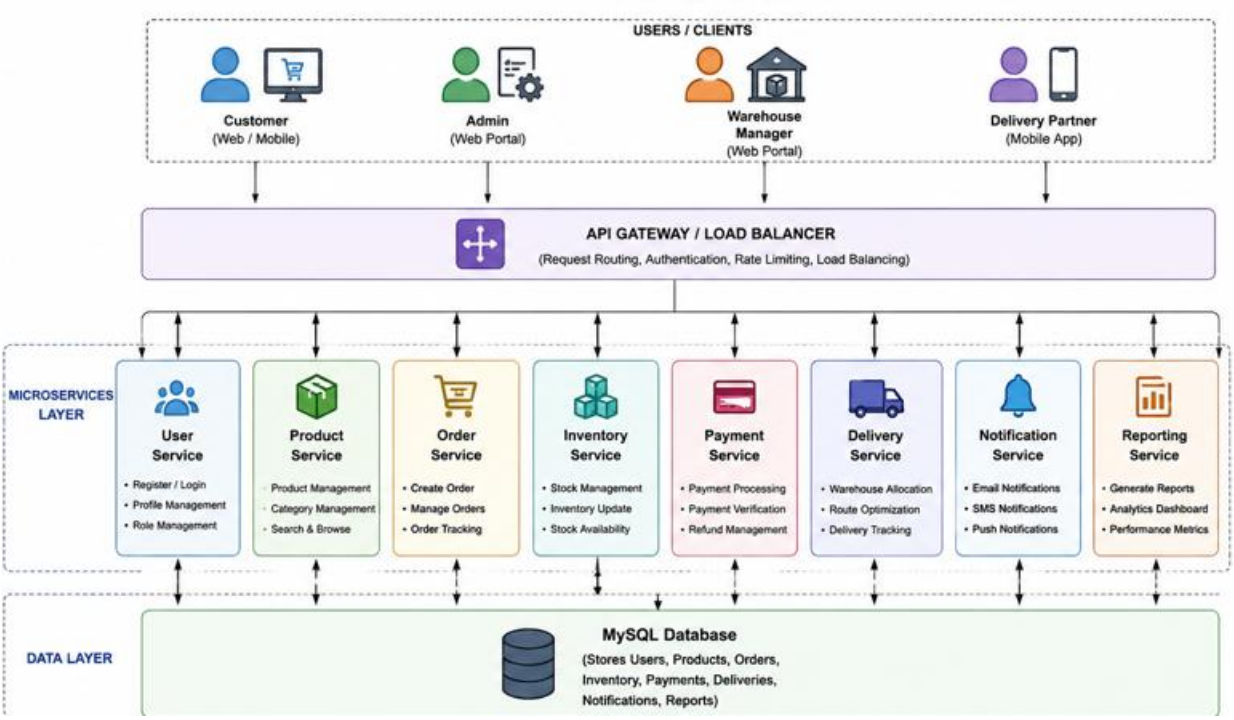
- The quality attributes influencing the architecture are scalability to handle increasing customer orders, reliability for continuous service, security for protecting customer and payment data, performance for fast order processing, maintainability for easy updates, and availability to provide uninterrupted services.

2. Selection of Software Architecture

- A Microservices Architecture is selected because it divides the application into independent services such as User Service, Product Service, Order Service, Inventory Service, Payment Service, Delivery Service, Notification Service, and Reporting Service.
- Each service performs a specific function and communicates through REST APIs. This architecture is highly suitable because the platform processes a large number of customer orders simultaneously and requires independent scaling of different services.
- It also allows developers to update or deploy one service without affecting the others, making the system more flexible and easier to maintain.

3. Software Architecture Diagram

Component	**Responsibility**
Web/Mobile Application	Provides user interface for customers and administrators
API Gateway	Routes requests to appropriate services
User Service	Manages user registration and login
Product Service	Manages product information
Order Service	Processes customer orders
Inventory Service	Maintains stock availability
Payment Service	Processes online payments
Delivery Service	Manages shipment and delivery



4. Components and Their Interactions

- The Web or Mobile Client allows customers and administrators to access the system. The API Gateway receives all requests and forwards them to the appropriate microservice.
- The User Service manages user authentication and profiles, while the Product Service maintains product information. The Order Service processes customer orders and communicates with the Inventory Service to verify stock availability.
- After successful payment verification through the Payment Service and Payment Gateway, the Delivery Service assigns logistics partners and updates delivery status.
- The Notification Service sends email or SMS updates to customers, while the Reporting Service generates operational reports for administrators. All services store and retrieve information from the central MySQL database through secure APIs.

5. Quality Attribute Analysis

- The proposed Microservices Architecture supports scalability by allowing each service to scale independently according to demand. It improves security through secure payment transactions.
- The architecture provides high performance because services can process requests simultaneously. It ensures reliability by isolating failures so that one service failure does not affect the entire system.
- Maintainability is improved because each service can be updated independently, while availability is achieved through cloud deployment, load balancing, and service redundancy.

Scalability

The system is designed using a microservices architecture, allowing individual services such as order management, inventory, and payment to scale independently. This enables the platform to handle increasing numbers of users and customer orders efficiently.

Security

The system ensures secure user authentication, role-based access control, encrypted data storage, and secure payment processing through trusted payment gateways. HTTPS communication protects sensitive customer and payment information.

Performance

The platform provides fast response times by processing multiple requests simultaneously. Load balancing and optimized database queries improve order processing speed and reduce delays during peak shopping periods.

Reliability

The architecture ensures reliable operation by isolating failures within individual services. Regular data backups, error handling, and recovery mechanisms help maintain continuous system functionality and accurate order processing.

Maintainability

The system is divided into independent services, making it easier to update, test, and maintain individual components without affecting the entire application. This reduces maintenance time and simplifies future development.

6. Architectural Justification

The Microservices Architecture is chosen because it supports the large-scale nature of e-commerce applications. It provides flexibility, faster deployment, and easier maintenance compared to a monolithic architecture. Although microservices increase deployment complexity and require API communication between services, the benefits of scalability, fault isolation, and independent development outweigh these challenges. The architecture also supports future enhancements such as AI-based demand forecasting, recommendation systems, chatbot integration, real-time analytics, and IoT-enabled warehouse management without major changes to the existing system.